GROUPE 6 · TECHNOLOGIES BIG DATA AVANCÉES 2026

Pipeline de recherche sémantique sur documents (RAG)

Wikipédia en français · Airbyte · Prefect · PostgreSQL · Qdrant · Ray · Gradio

Salma Abarkane · Rania Bouaroua · Salma Benmouma · Oumaima Gassi · Imad Eddine El Khorassani · Mohammed Yassine Benmeziane · Anys Louzal · Mohamed Anouar Faress

Le RAG : répondre en citant ses sources

Un modèle de langage seul peut inventer. Le RAG lui fournit d'abord les bons passages, puis lui demande de répondre uniquement à partir d'eux.



1. Question

L'utilisateur pose une question en langage naturel dans l'application Gradio.



2. Recherche

La question est vectorisée, puis Qdrant renvoie les passages les plus proches parmi 14 290.



3. Génération

Le LLM rédige la réponse à partir de ces passages seulement, et cite ses sources [1], [2]…

Le défi Big Data : préparer et indexer des millions de passages avant la première question.

Le corpus : Wikipédia en français

2,6 M

articles dans le corpus complet

17

fichiers parquet, environ 5 Go compressés

112 313

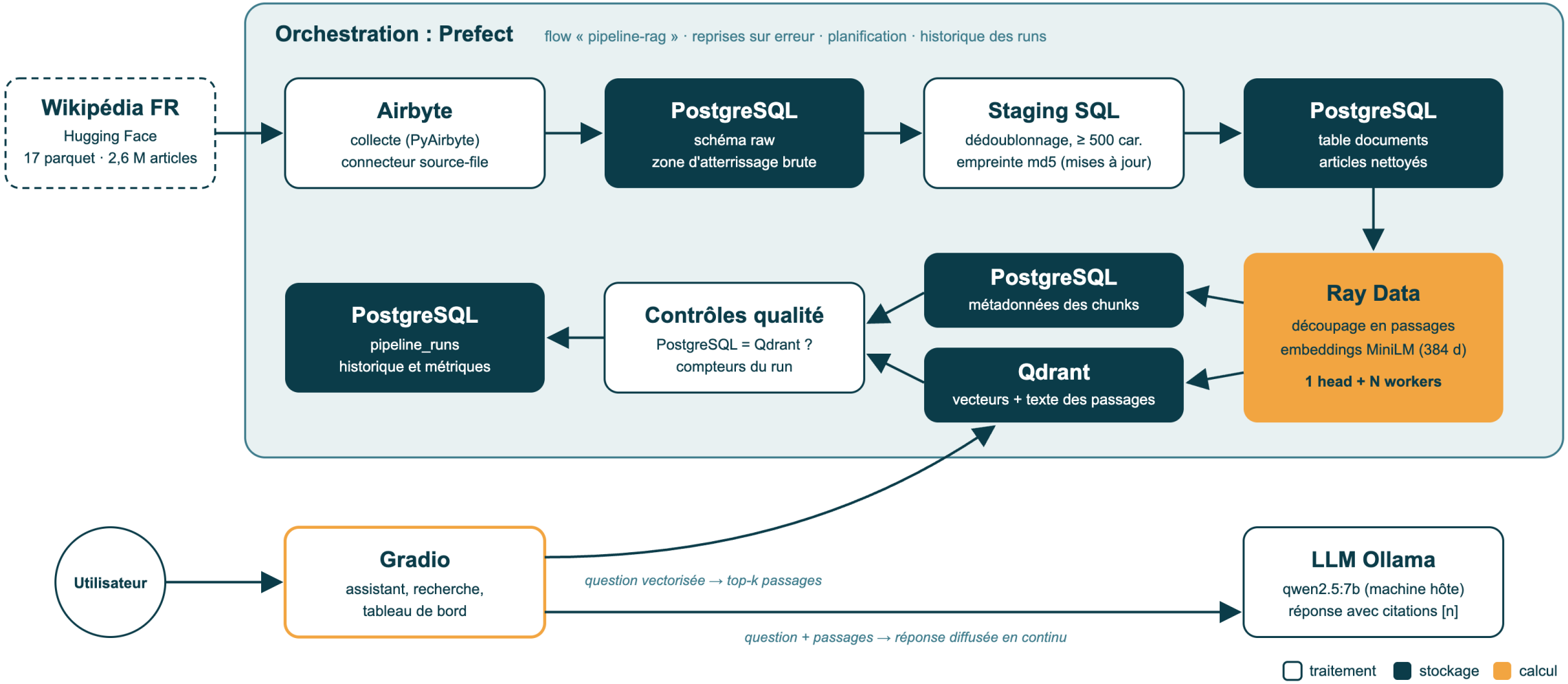
articles retenus dans le fichier 16, sur 150 861 lus

14 290

passages vectorisés, issus de 2 000 articles

Source : jeu de données wikimedia/wikipedia, version 20231101.fr, sur Hugging Face (licence CC BY-SA). Le staging garde les articles d'au moins 500 caractères, sans doublon : 74 % du fichier 16.

Architecture du pipeline



Collecte et staging : Airbyte + PostgreSQL

Airbyte

- PyAirbyte et le connecteur officiel source-file (parquet en HTTPS)
- Écriture dans le schéma raw de PostgreSQL, en mode ajout
- Connecteur isolé en Python 3.11, préinstallé dans l'image
- 2 reprises automatiques en cas d'échec

Staging SQL

- Une seule requête : DISTINCT ON et filtre ≥ 500 caractères
- Empreinte md5 : seuls les articles modifiés sont réécrits
- Leurs anciens passages sont supprimés, puis revectorisés
- Zone brute vidée après chaque run

150 861 articles lus → **112 313** articles retenus · collecte et staging du fichier 16 en **~2 min**

Traitement distribué : Ray Data

1 **Lecture et découpage**
tâches de 200 articles · 0,5 CPU chacune

2 **Embeddings**
1 acteur de 2 CPU par worker · modèle chargé une seule fois

3 **Écriture**
vecteurs dans Qdrant · métadonnées dans PostgreSQL

Exécution en flux : les premiers passages sont écrits pendant que les suivants sont encore calculés.

Le cluster

- 1 head qui ne fait que coordonner
- N workers de 3 CPU et 2 Go
- Modèle MiniLM multilingue, 384 dimensions, sur CPU (FastEmbed)
- Passages de 500 caractères, 75 de recouvrement
- Identifiants déterministes : relancer ne crée aucun doublon

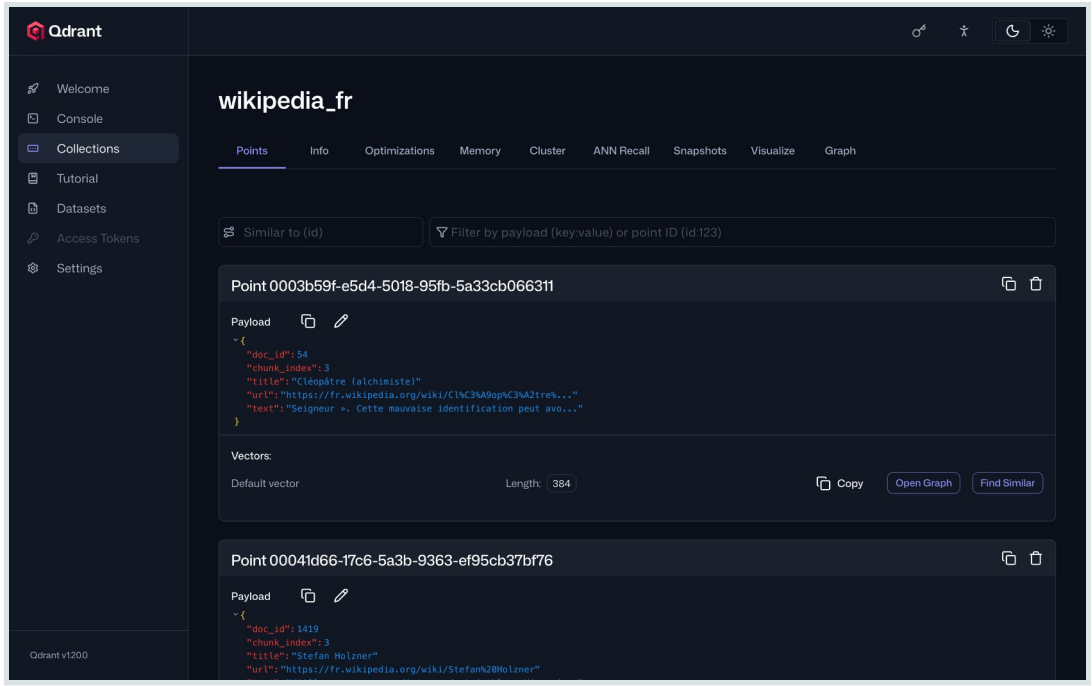
Qdrant : la base vectorielle

1 point = 1 passage
vecteur de 384 dimensions + titre, URL et texte

Quantification int8
4 fois moins de mémoire ; les meilleurs candidats sont recalculés exactement

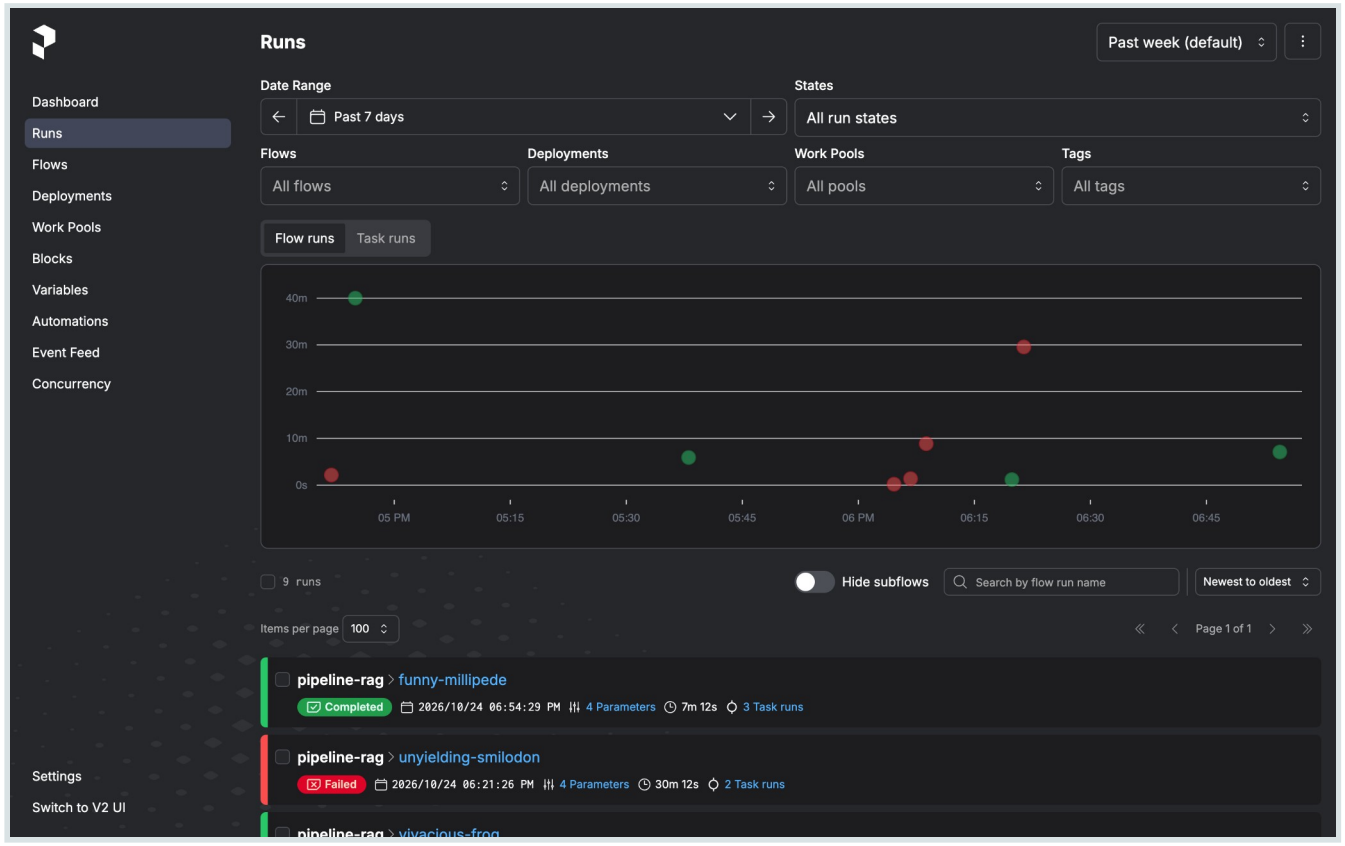
Index de charge utile
doc_id pour les suppressions, titre pour le filtre plein texte

140 ms
durée médiane d'une recherche, vectorisation de la question comprise



Un point de la collection wikipedia_fr

Orchestration : Prefect



1 **collecte-airbyte** · 2 reprises automatiques

2 **staging-postgres** · dédoublonnage et filtre

3 **vectorisation-ray** · job soumis au cluster Ray

4 **controles-qualite** · PostgreSQL = Qdrant ?

Déploiement planifié chaque nuit à 2 h, paramètres modifiables depuis l'interface, chaque run tracé dans pipeline_runs.

L'application Gradio



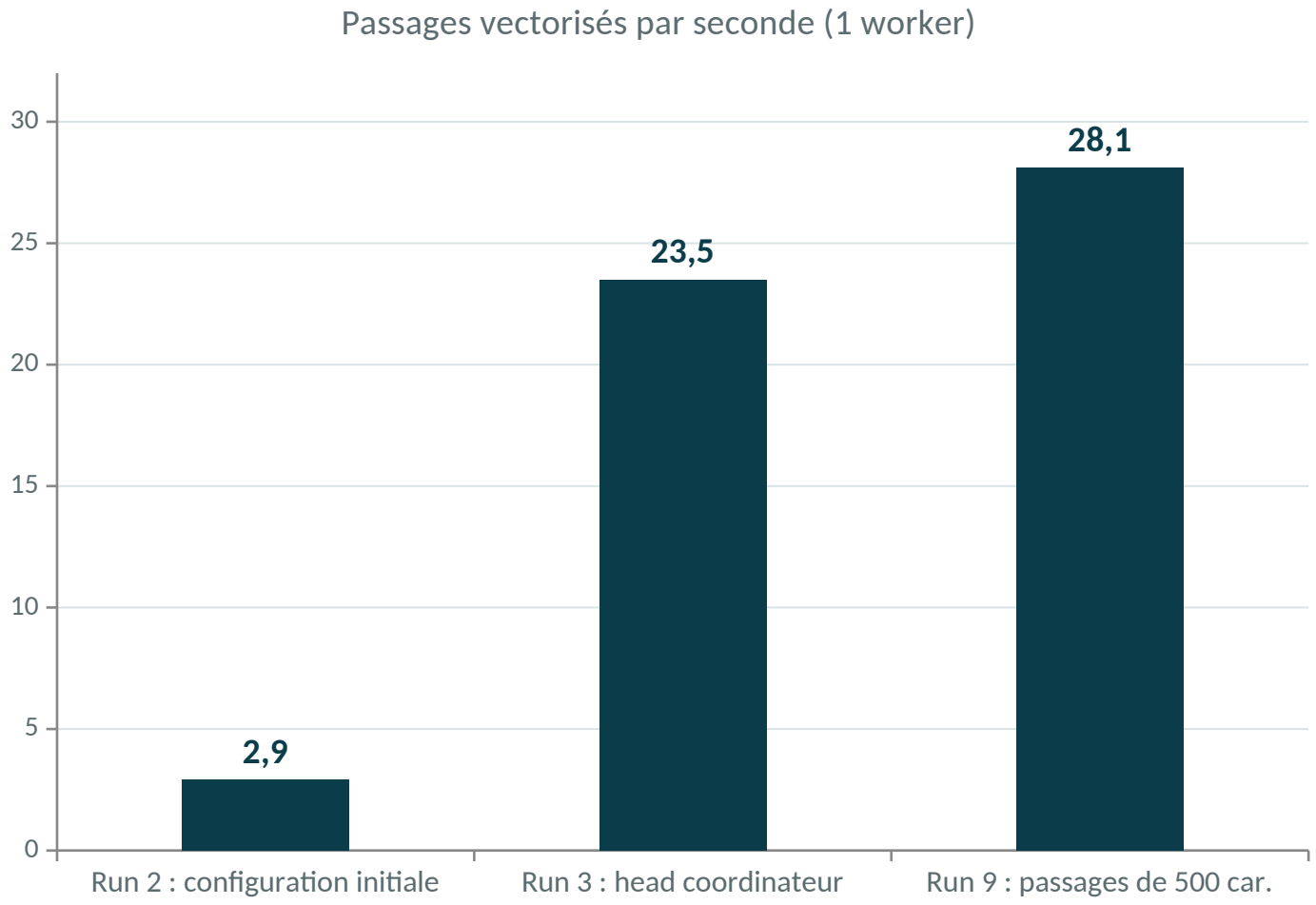
Assistant
réponse diffusée au fil de l'eau, sources avec score et lien

Recherche sémantique
passages les plus proches, filtre sur le titre

Pipeline
volumes, derniers runs et questions posées

Chaque question est enregistrée : scores, durées et sources.

Optimiser le cluster : débit multiplié par 9,7



Head coordinateur seul
Il exécutait aussi des tâches, et l'écriture a été tuée 41 fois.

3 CPU par worker
Lecture et découpage fusionnés en une seule tâche.

Passages adaptés au modèle
500 caractères au lieu de 1 000 : plus rien n'est tronqué.

À ce débit, le fichier 16 complet prendrait environ 8 h avec 1 worker.

Qualité de la recherche

86,7 %

Hit@1 : bon article en 1re position

93,3 %

Hit@5 : bon article dans les 5 premiers

0,90

MRR@10, sur une échelle de 0 à 1

140 ms

durée médiane d'une recherche

Méthode : 60 questions écrites par le LLM à partir de passages tirés au hasard ; on vérifie si la recherche retrouve l'article d'origine. 57 articles sur 60 retrouvés dans le top 10. Les 5 réponses générées citent leurs sources, en 7,9 s (médiane).

À nuancer : questions proches du texte et petit index (14 290 passages) ; à confirmer avec des questions écrites à la main.

Difficultés rencontrées et solutions

Run 2 : 2 445 s pour 2 000 articles

Solution : Head réservé à la coordination et 3 CPU par worker : 356 s.

Ray tuait des tâches à tort

Solution : Il lisait la mémoire de toute la VM : limite par conteneur et mémoire fixée au démarrage.

Passages tronqués par le modèle

Solution : MiniLM ne lit que 128 tokens : passages ramenés à 500 caractères.

Manque de mémoire de la VM Docker

Solution : Diagnostic via memory.events ; nombre de workers rendu paramétrable.

Limites et perspectives

Limites

- 2 000 articles indexés sur 2,6 millions
- Passage à l'échelle non mesuré (8 Go de mémoire Docker)
- Évaluation sur 60 questions synthétiques
- Environ 8 s par réponse sur un ordinateur portable

Perspectives

- Cluster Ray sur plusieurs machines (KubeRay) et mesure de 1 à 8 workers
- Embeddings sur GPU
- Recherche hybride et re-classement des résultats
- Jeu de questions écrites à la main
- Mise à jour avec les nouveaux dumps Wikipédia

L'architecture est prête : ajouter des workers, des fichiers du corpus ou de nouvelles sources ne demande que de changer des paramètres.

L'équipe

SA

Salma Abarkane
Cheffe de projet, architecture

RB

Rania Bouaroua
Collecte · Airbyte

SB

Salma Benmouma
Orchestration · Prefect

OG

Oumaima Gassi
Stockage · PostgreSQL

IE

Imad Eddine El Khorassani
Stockage NoSQL · Qdrant

MY

Mohammed Yassine Benmeziane
Traitement · Ray

AL

Anys Louzal
Visualisation · Gradio

MA

Mohamed Anouar Faress
RAG et évaluation

MERCI POUR VOTRE ATTENTION

Place à la démo

Gradio : localhost:7860 · Prefect : localhost:4200 · Ray : localhost:8265 · Qdrant : localhost:6333/dashboard

Dépôt : github.com/Anyslouzal/projet-rag-bigdata